Exploring Cloud Microphysics Modeling Concepts the Pythonic Way

Agnieszka Żaba, Sylwester Arabas

ZFŚ Seminar, 22.10.2026

OPENATMOS
ENVIRONMENTAL PHYSICS GROUP
AGH
AGH UNIVERSITY OF KRAKOW

https://doi.org/10.5194/egusphere-2026-3906
Preprint. Discussion started: 21 July 2026
© Author(s) 2026. CC BY 4.0 License.

Exploring Cloud Microphysics Modeling Concepts the Pythonic Way

Sylwester Arabas[1], Agnieszka Żaba[1], Daria Klimaszewska[1], Emma C. Ware[2], Clare E. Singer[3], Manhal Alhilali[4], Jason Barr[5], Daniel G. Partridge[6], Shin-ichiro Shima[4], and Robert Wood[5]

[1]Faculty of Physics and Applied Computer Science, AGH University of Krakow, Krakow, Poland
[2]Department of Land, Air, and Water Resources, University of California Davis, Davis, CA, USA
[3]University of Colorado, Boulder, CO, USA
[4]Graduate School of Information Science, University of Hyogo, Kobe, Japan
[5]Department of Atmospheric and Climate Science, University of Washington, Seattle, WA, USA
[6]Department of Mathematics and Statistics, University of Exeter, Exeter, UK

Correspondence: Sylwester Arabas (sylwester.arabas@agh.edu.pl) and Agnieszka Żaba (azaba@agh.edu.pl)

scope of the paper and this talk

Introduction

1. aerosol size spectrum sampling
2. unit-aware coding and plotting
3. parcel model governing equations
4. **JIT compilation using Numba**
5. parcel ODE system definition
6. test-time dimensional analysis using Pint
7. initial condition at equilibrium
8. ODE integration and result analysis
9. Super-Droplet Method algorithm
10. SDM against analytic solution

Summary

Code availability



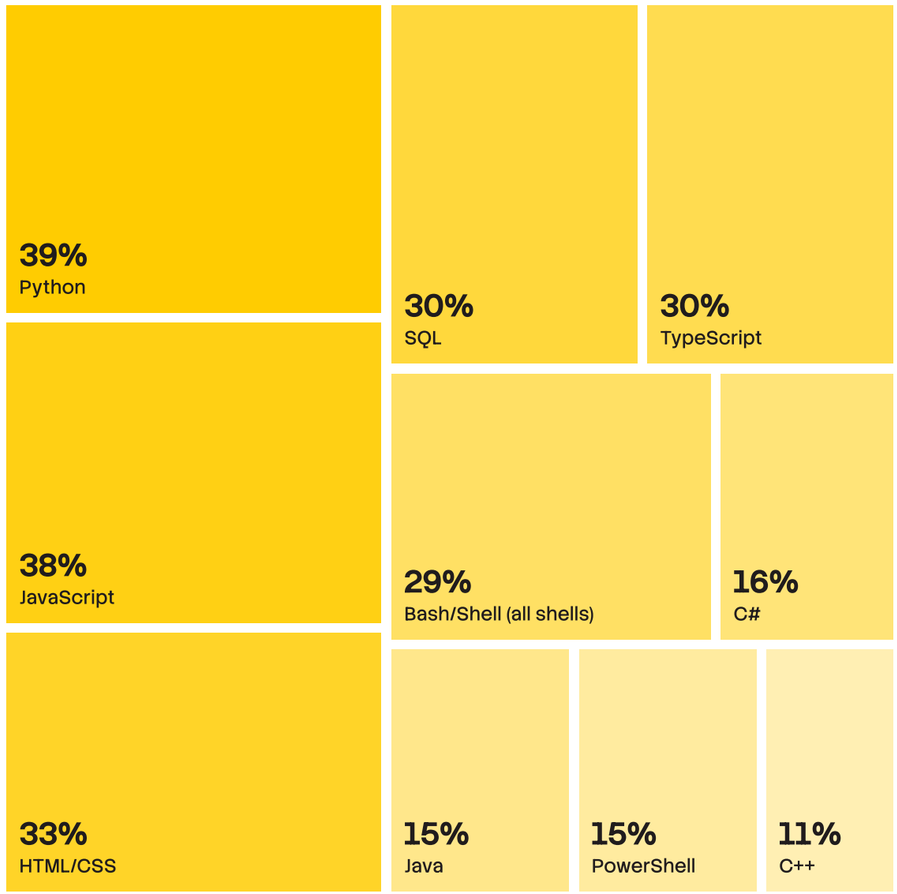




`Python` and modeling?

The core of the language does not support:

- ordinary int
- multi-dimensional arrays
- JIT compilation
- multi-threading
- GPUs
- numerical algorithms





Technology

Most popular technologies

Top programming languages used for AI shows Python leading at 39% usage

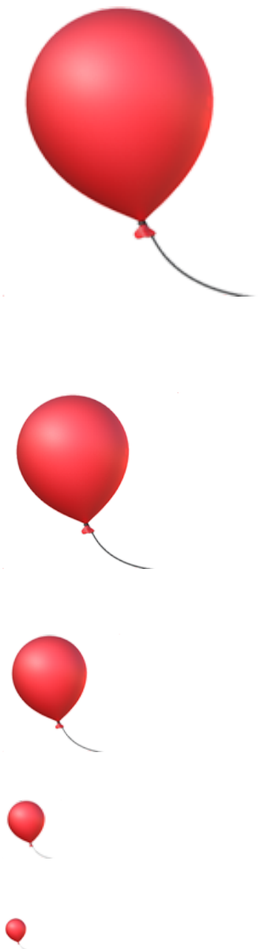
But `Python` is designed as a **glue language**!

Above features are handled by packages: **NumPy**(C), **SciPy**(C/Fortran/…), **Numba/JAX**(C++/…), and many more!

unit-aware coding and plotting

section 2 in paper

a useful physical example: ideal gas



equation of state linking

| : density
| : pressure
| : temperature

ρ
p
T

$$\rho = (pM)/(TR)$$

where

| : molar mass
| : universal gas constant

M
R

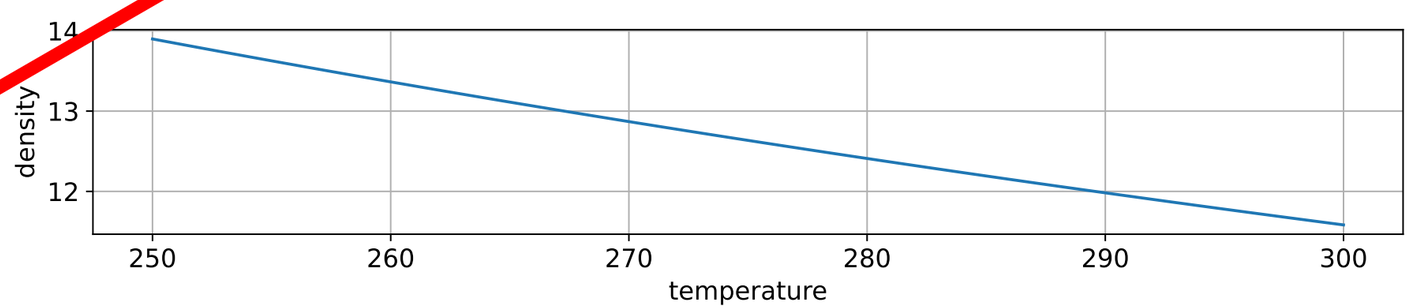
Issue: units in code (example: ideal gas density)

$$\rho = (pM)/(TR)$$

```
%config InlineBackend.figure_formats = ['svg']
import numpy, scipy, matplotlib.pyplot as plt
import mendeleev
```

```
p = 1000                                    # pressure [mbar]
T = numpy.linspace(250, 300)                # temperature [K]
R = scipy.constants.R                       # gas constant [J/K/mol]
M = (0.78 * 2 * mendeleev.N.atomic_weight +  # molar mass [g/mol]
     0.22 * 2 * mendeleev.O.atomic_weight)
rho = (p * M) / (R * T)

fig, axs = plt.figure(), plt.gca()
axs.plot(T, rho)
axs.set(ylabel="density", xlabel="temperature")
```

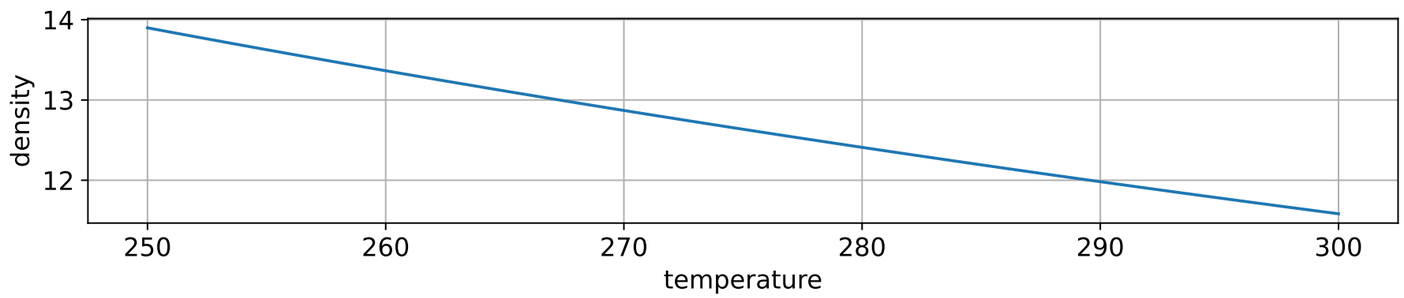


"*The proper use of comments is to compensate for our failure to express ourself in code … Comments are always failures.*" (Robert C. Martin *Clean Code*, 2009)

Issue: units in comments not in code

The interpreter is not parsing the comments! We always use units on paper, use it in code as well.

- What is the unit of the plotted data?



- Reading the code, one needs to go to the definition each time.

```
1 print(T[0])
```

```
250.0
```

is time or temperature?

T

- No way of enforcing changes in usage upon change of definition.
- After changing the value — did you change the comment? (We all know the answer…)

physical units in code with `Pint`

Example Gallery

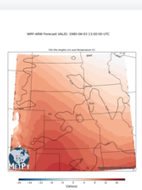
For more MetPy examples, please visit the MetPy Cookbook on Project Pythia.

General Examples

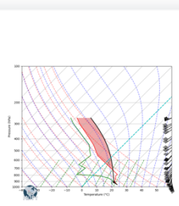
Examples of using a variety of MetPy's functionality together.



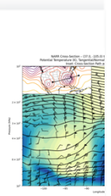
XArray Projection Handling



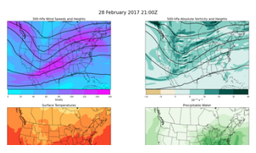
Sigma to Pressure Interpolation

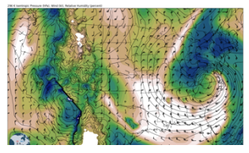


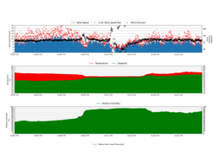
Advanced Sounding

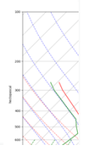


Cross Se
Analysis









What is Pint?

Contents

- Design principles
- One last thing
- License

Pint is a Python package to define, operate and manipulate **physical quantities**: 1
of a numerical value and a unit of measurement. It allows arithmetic operations
them and conversions from and to different units.

It is distributed with a comprehensive list of physical units, prefixes and constant
modular design, you can extend (or even rewrite!) the complete list without chan
source code. It supports a lot of numpy mathematical operations **without monk
or wrapping numpy**.

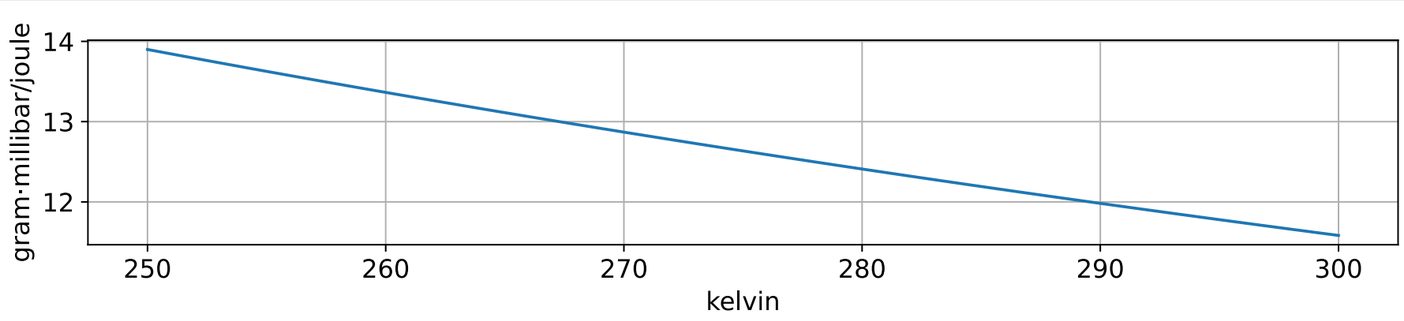
physical units with Pint (example: ideal gas density)

$$\rho = (pM)/(TR)$$

```
import pint
SI = pint.UnitRegistry()
SI.setup_matplotlib()

pressure = 1000 * SI.mbar
temperatures = numpy.linspace(250, 300) * SI.K
gas_const = scipy.constants.R * SI.J / SI.K / SI.mol
molar_mass = (0.78 * 2 * mendeleev.N.atomic_weight +
              0.22 * 2 * mendeleev.O.atomic_weight) * SI.g / SI.mol

density = (pressure * molar_mass) / (gas_const * temperatures)
plt.plot(temperatures, density)
```

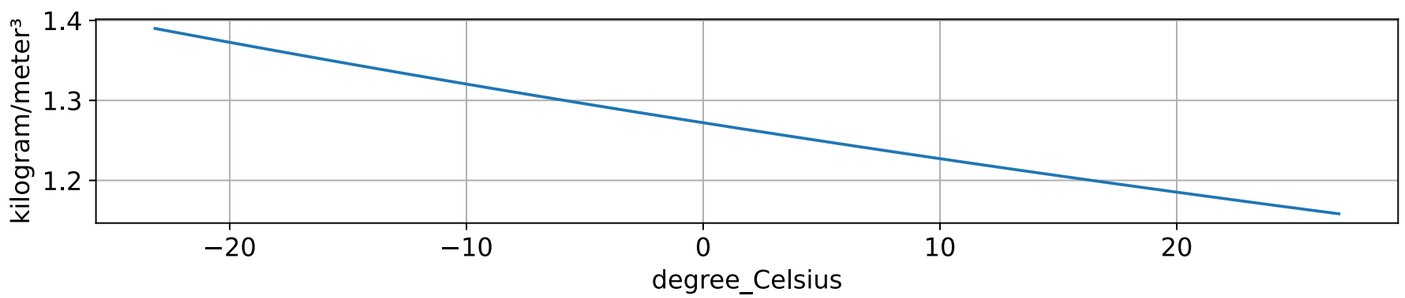


- changed names of variables
- axis labels automatic

Can we …

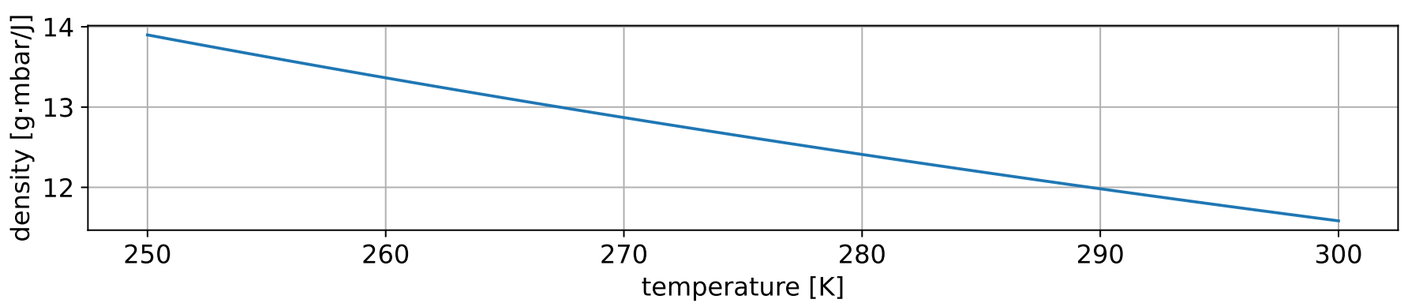
- … change units on plot?

```
1 fig, axs = plt.figure(), plt.gca()
2 axs.xaxis.set_units(SI.degC)
3 axs.yaxis.set_units(SI.kg / SI.m**3)
4 axs.plot(temperatures, density)
```



- … change axes labels and still use units?

```
1 fig, axs = plt.figure(), plt.gca()
2 axs.plot(temperatures, density)
3 axs.set_xlabel(f"temperature [{axs.xaxis.units:~P}]")
4 axs.set_ylabel(f"density [{axs.yaxis.units:~P}]");
```



dimensional analysis using Pint

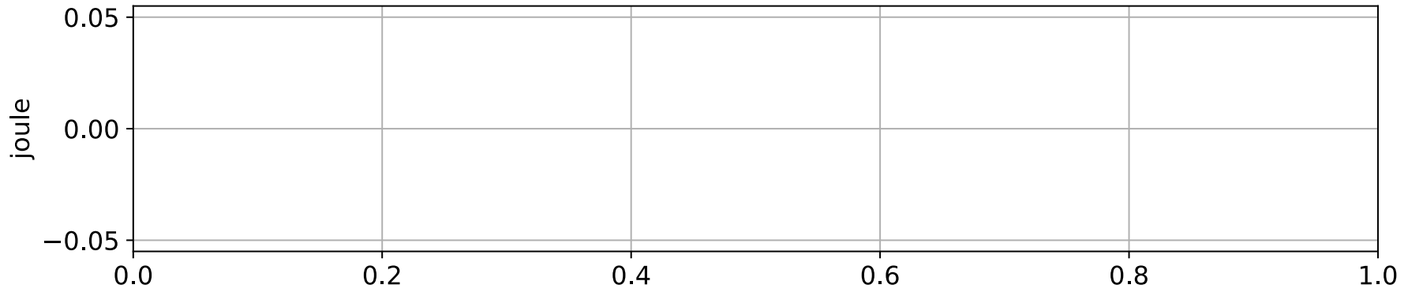
section 6 in paper

Can we …

- … change the units to the wrong ones?

```
1 plt.gca().yaxis.set_units(SI.J)
2 plt.plot(pressure)
```

```
ConversionError: Failed to convert value(s) to axis units: <Quantity([1000], 'millibar')>
```



- … put different quantities on same axis?

```
1 try:
2     plt.plot(temperatures)
3     plt.plot(temperatures ** 2)
4 except Exception as e:
5     plt.close()
6     print(e.__cause__)
```

```
Cannot convert from 'kelvin ** 2' ([temperature] ** 2) to 'kelvin' ([temperature])
```

Can we …

- … add Kelvins to Pascals?

```
1 temperatures + pressure
```

```
DimensionalityError: Cannot convert from 'kelvin' ([temperature]) to 'millibar' ([mass] / [length] / [time] ** 2)
```

- Tools for unit testing:

```
1 ((pressure * molar_mass) / (gas_const * temperatures)).check("[density]")
```

```
True
```

```
1 assert density.check("[density]")
```

Pint: key takeaways

- ⚖️ performs dimensional analysis (unit testing!)
- ⚙️ automates plot axis labelling (and consistency checks)
- 💱 handles SI prefixes and unit conversions
- 🩺 helps in debugging (`print(x)` prints a value with unit)

Pint: key (alleged) limitations

- ⏱️ incurs performance overhead
- 🔗 no out-of-the-box interoperability with SciPy/Numba

Issue: units with SciPy and Numba

Problem with using units with SciPy/NumPy packages:

```
1 from scipy.optimize import root_scalar
2
3 f = lambda T, a, b: a / T + b
4
5 root_scalar(
6     f=f,
7     x0=1 * SI.K,
8     args=(2 * SI.K**2, -3 * SI.K)
9 )
```

```
DimensionalityError: Cannot convert from 'kelvin ** 2' ([temperature] ** 2) to 'kelvin' ([temperature])
```

Problem with using just-in-time compilation (Numba)

```
1 import numba
2
3 @numba.jit
4 def f(a, b, T):
5   return a / T + b
6
7 print(f"{f(2, -3, 270):.3g}")
```

```
-2.99
```

```
1 f(2 * SI.K**2, -3 * SI.K, 270 * SI.K)
```

```
TypingError: non-precise type pyobject
During: typing of argument at /var/folders/gl/1t3c1h9x3ws6d68b11gr2rth0000gn/T/ipykernel_58840/2583992666.py (3)
```

Using `Pint` with `SciPy` solvers or with `Numba` JIT compiler

(`see Section 4. in the preprint`, based on BEng thesis of Daria Klimaszewska)

injectable constants-catalogue idiom

💡 key (physics) principle: dimensionality of physical quantities originates from **constants** only!

❶ instantiate a constants catalogue at **topmost scope**, pass to all physics-related functions

❷ ~~⇝~~ use **unit-aware** constants catalogue for unit testing

❸ ~~⇝~~ use **plain-floats** constants catalogue elsewhere

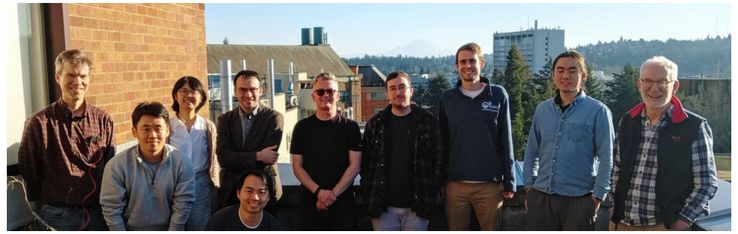
⇝

- ✅ no trade-off in **code readibility** or **performance**
- ✅ no trade-off in **functionality** (e.g., SI prefixes, unit conversions, …)
- ✅ no trade-off in **interoperability** with numerical routines or JIT compilers
- 💡 works with **any language** supporting JIT compilation (e.g., Julia) or templates (e.g., C++)
- ➕ positive side-effect: enforces **separation of concerns**

no excuse for coding without units, let’s teach it!

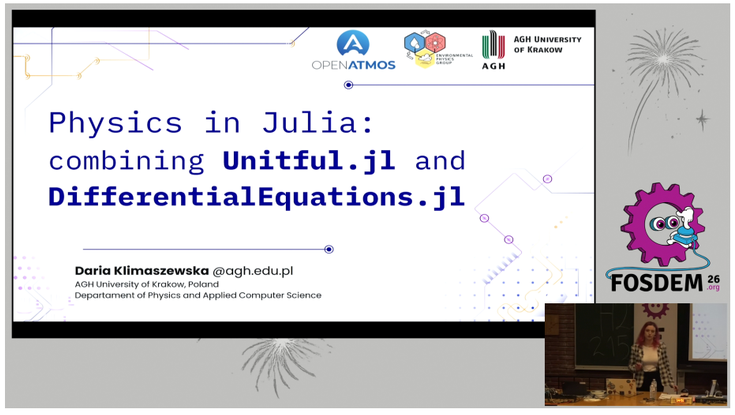
“hacking aerosol-cloud µ-physics modeling concepts in Python”

University of Washington, Jan 2026, Erasmus+ in Seattle



FOSDEM 2026 (Jan/Feb)

Université libre de Bruxelles



“Gluing SciPy, Numba, and Pint to Bridge High Performance with Maintainability”

EuroScipy 2026 (Jul, **Wydział Informatyki AGH**)



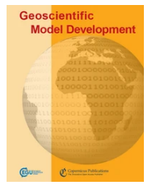
- **AGH courses**: PP, PI, MUP, MPS, RSE, MPTS, PTS…

From executable code to a reproducible scientific paper

PySDM

Idea

- Cloud microphysics as a case study
- Executable code embedded in the paper
- Ready-to-use workshop materials
- Reproducible results and figures



Venue

- Geoscientific Model Development (GMD)
- Review and perspective paper
- GMD's reproducibility requirements

jupyter

Tools

- Jupyter for executable code
- Quarto for paper generation
- Automated tests and reproducible outputs

One executable source Paper + Figures + Workshop

Reproducible by authors, editors, reviewers, and readers

For reproducible science use Quarto.org

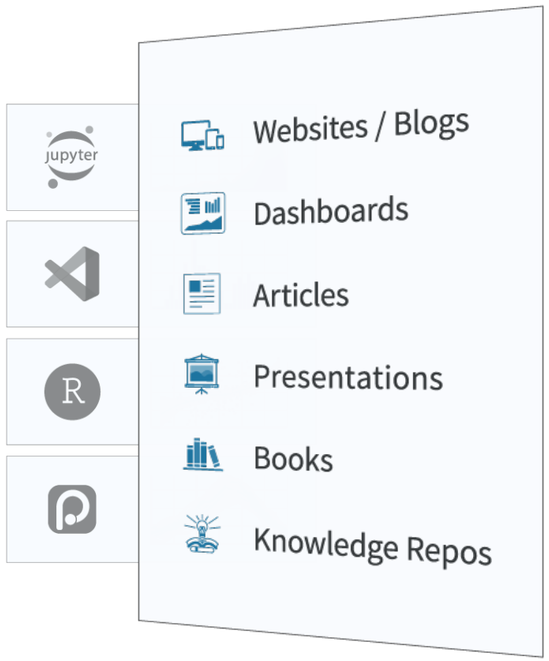
quarto Overview Get Started Guide Extensions Reference Gallery Blog Help

Welcome to Quarto

An open-source scientific and technical publishing system

- Author using Jupyter notebooks or with plain text markdown in your favorite editor.
- Create dynamic content with Python, R, Julia, and Observable.
- Publish reproducible, production quality articles, presentations, dashboards, websites, blogs, and books in HTML, PDF, MS Word, ePub, and more.
- Share knowledge and insights organization-wide by publishing to Posit Connect, Confluence, or other publishing systems.
- Write using Pandoc markdown, including equations, citations, crossrefs, figure panels, callouts, advanced layout, and more.

Analyze. Share. Reproduce. You have a story to tell with data—tell it with Quarto.



Get Started Guide

Get started

Code availability

For reproducible science use Quarto

```
quarto render paper.ipynb --execute --to copernicus-pdf
```

For reproducible science use `Quarto`

- useful for working on
  - articles, reports, theses, books (`pdf`, `epub`, …)
  - presentations, teaching materials (`revealjs`, `beamer`, …)
  - web pages, data access and quick look dashboards (`html`, …)

- supports
  - BibTeX
  - syntax
  - vector graphics

- save rendered `.tex`, `.md` files

```
keep-md: true
keep-tex: true
```

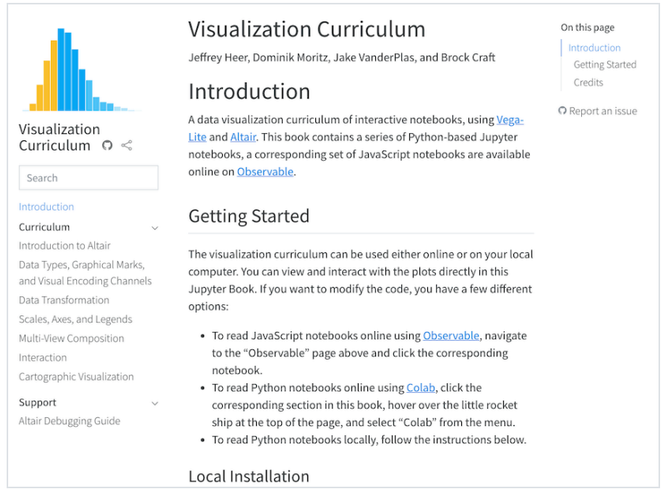
This presentation is in `Quarto`, including code execution!

Example gallery

Gallery

Quarto can produce a wide variety of output formats. Here are some examples:

- Articles & Reports
- Presentations
- Dashboards
- Websites
- Books
- Interactive Docs



Create multi-format books

Articles & Reports

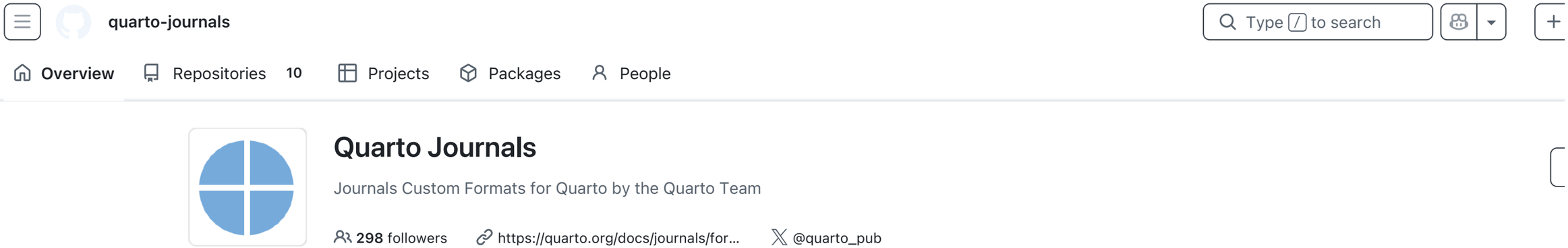
Write a single markdown file and create output in a variety of formats.

HTML for web publishing </>

PDF for high quality print </>

MS Word for Office workflows </>

Journal templates from major publishers



Quarto in Nature

nature

TECHNOLOGY FEATURE | 28 February 2022

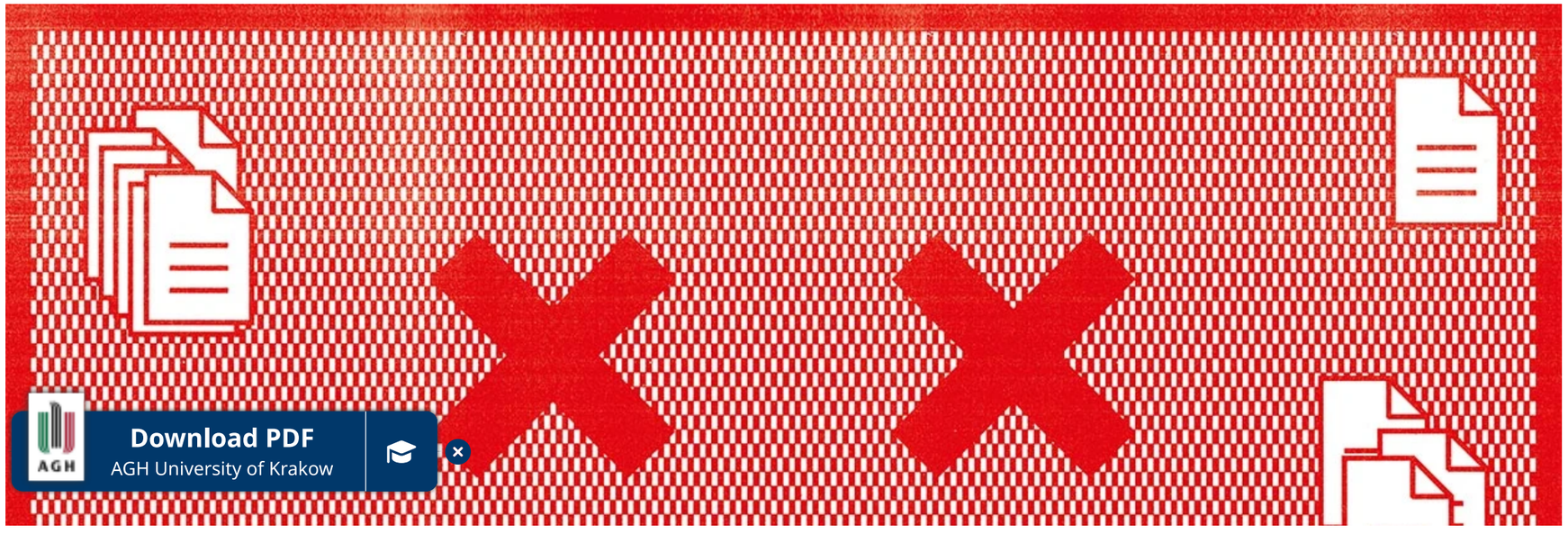
Cut the tyranny of copy-and-paste with these coding tools

‘Executable manuscripts’ insert results directly into documents, eliminating common mistakes.

By Jeffrey M. Perkel

Save article

View saved research ›

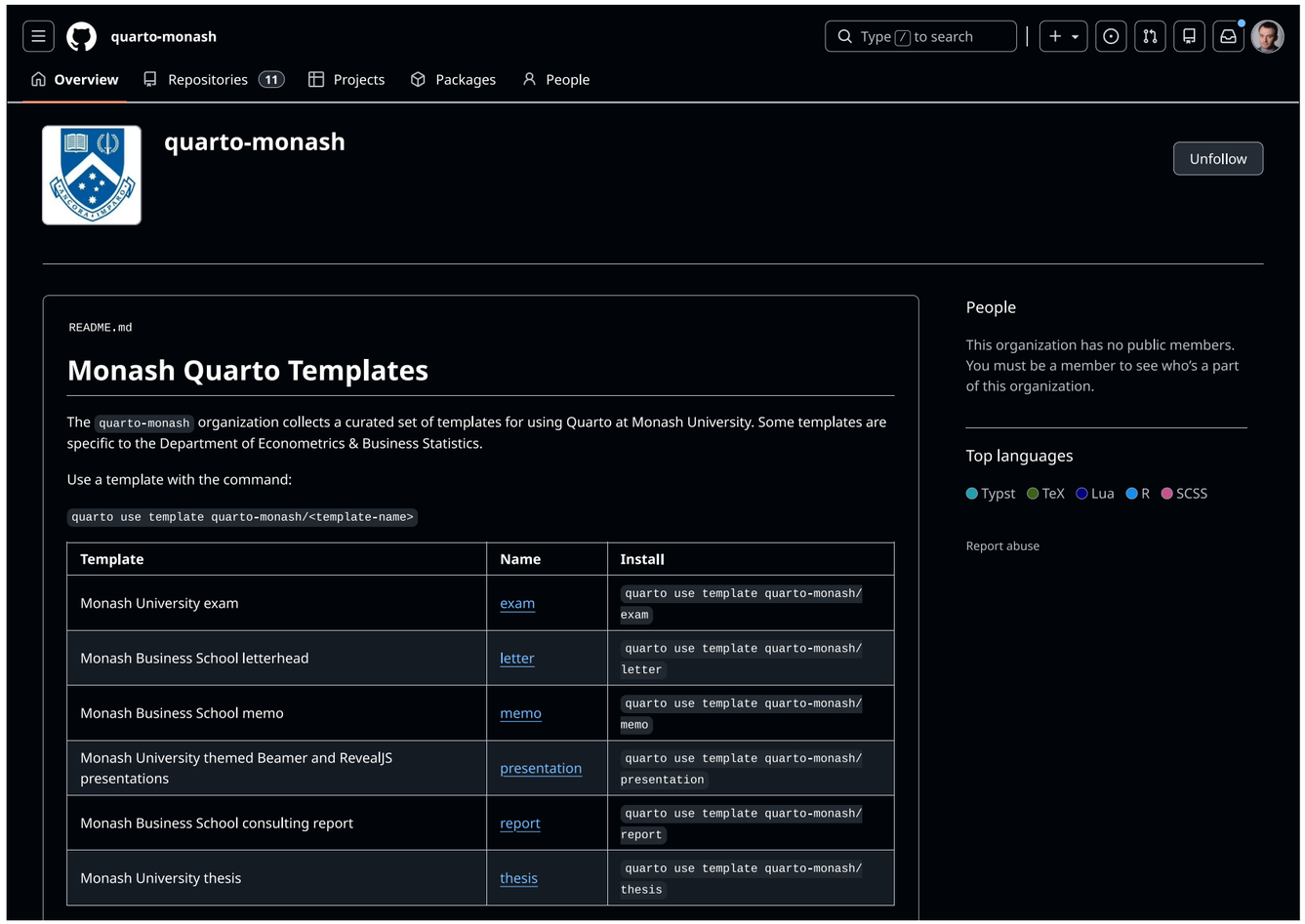


Download PDF
AGH University of Krakow

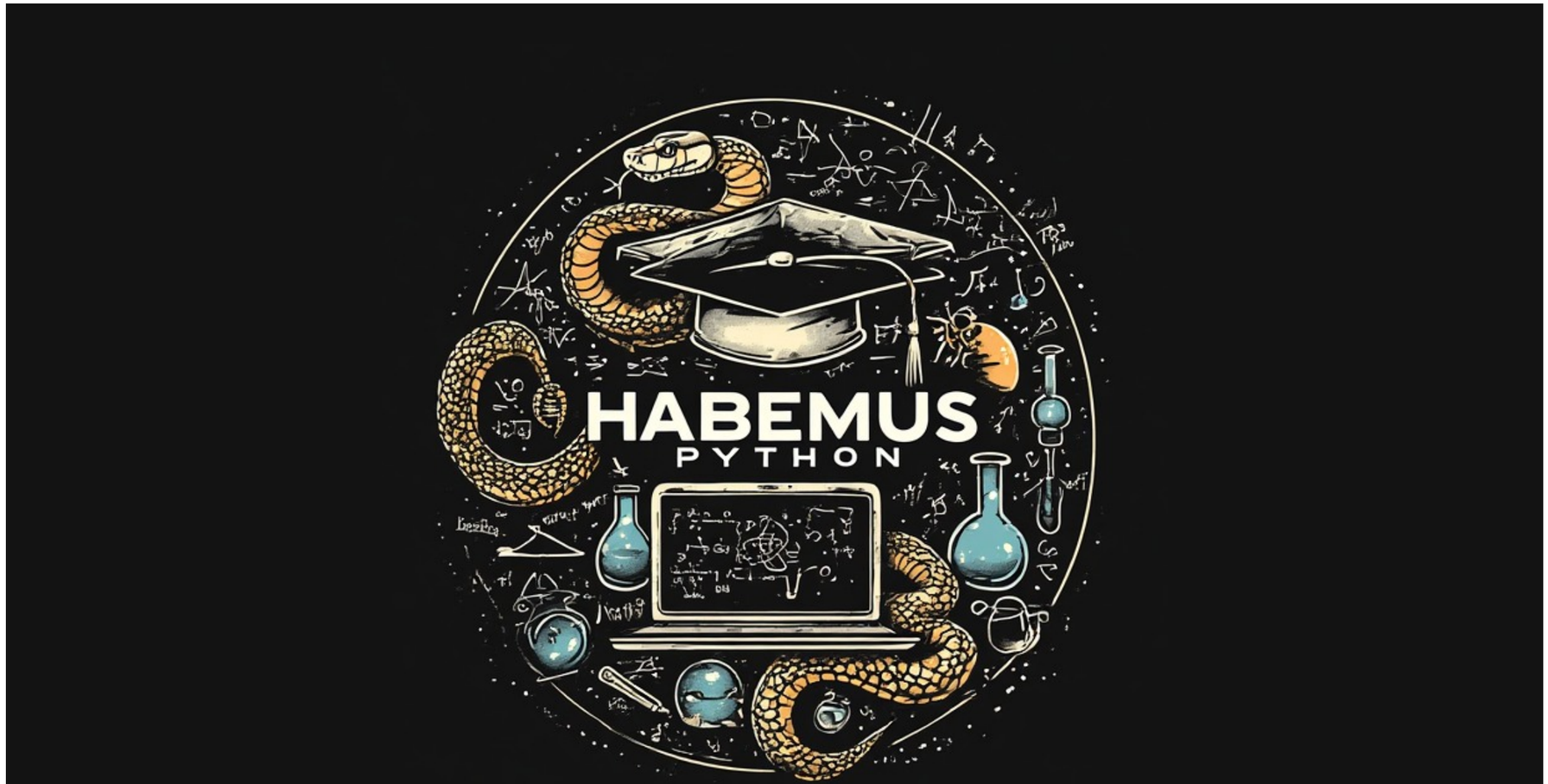
Quarto: Apt for BEng/MSc/PhD theses

University Quarto templates (theses, slides, letters)

- 🇦🇺 monash.edu (Melbourne)
- 🇳🇱 uu.nl (Utrecht)
- 🏴 ed.ac.uk (Edinburgh)
- 🇬🇧 warwick.ac.uk (Warwick)
- 🇦🇺 mq.edu.au (Sydney)
- 🇺🇸 gatech.edu (Atlanta, GA)
- 🇩🇰 au.dk (Aarhus)
- 🇺🇸 unm.edu (Albuquerque, NM)



Habemus Python https://habemus-python.github.io



Handbook of Automation for BEng & Master's projects USing PYTHON

We create resources to help students combine reproducible research, software engineering with CI automation, and high-quality, code-based typesetting. Our practical examples show how to use the "executable manuscripts" approach (see, e.g., Perkel 2022, Nature).

Write it. Run it. Share it. Engineer your documents to be reproducible, automated, and aesthetic by design!

Habemus Python https://habemus-python.github.io

1.4. Organising code/text review workflow with advisors and collaborators

This section is for mentor–mentee pairs of all kinds – whether the mentor already uses GIT, QUARTO or pull requests. The point is simple: a review workflow built on GIT and GITHUB is not hard, and it makes supervising a thesis simpler, safer and faster. It is optional and useful even for a paper-only thesis, as it keeps a time-stamped record of who changed what, when and why.

One change, one pull request. A pull request (PR) is a self-contained, reviewable proposal – a new subsection, a corrected figure, an updated derivation. Keeping each PR to a single logical change makes it easy to read, to discuss and to undo. Because the thesis is reproducible from its source, the reviewer can judge both *what changed in the source* and *how the rendered document now looks*.

Clean diffs. A JUPYTER notebook also stores execution counters, outputs and metadata; committing these buries the real edits in noise. The NB-CLEAN tool strips them, so a diff shows only meaningful changes – text, code and plotting cells:

```
nb-clean check thesis.ipynb   # report a dirty notebook (e.g. on CI)
nb-clean clean thesis.ipynb   # strip outputs, counts and metadata
```

Running it as a PRE-COMMIT hook keeps every commit clean automatically.

Reviewing on GITHUB. The mentee requests a review; the mentor comments on individual lines, may suggest concrete edits and chooses *Approve* or *Request changes*; the mentee pushes follow-up commits; and each thread is *resolved* once handled. Nothing is merged until the conversation has settled (Figure 1).

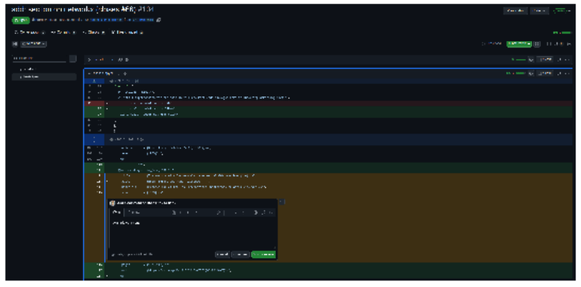
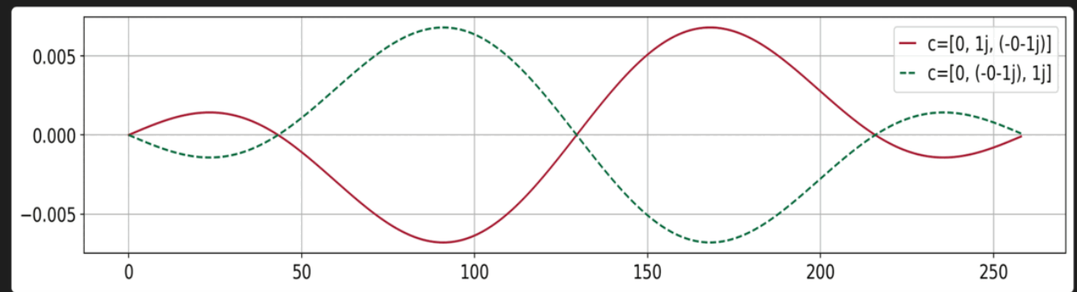


Figure 1: Reviewing a pull request on GITHUB: a notebook source diff with an inline review comment and the review controls (Comment, Approve, Request changes).

SLIDE WITH CODE AND A PLOT

```
1 import numpy as np, matplotlib.pyplot as plt
2 plt.plot(np.fft.ifft((c := [0, 1j, -1j]) + [0]*2**8).real, label=f'{c=}')
3 plt.plot(np.fft.ifft((c := [0, -1j, 1j]) + [0]*2**8).real, label=f'{c=}')
4 plt.legend()
```



Summary and outlook

- ⚖️ **Pint** (or Boost.Units in C++, UnitFull.jl in Julia, …):
  no excuse to write code without units.
- 📝 **Quarto**:
  highly encourage to embrace it with your mentees and mentors!
- 🌨️ **cloud/aerosol-μ-physics preprint** in public peer review in GMD:
  DOI: 10.5194/egusphere-2026-3906 (Colab-ready Quarto/Jupyter notebook as suppl.)
- 💡 idea: **workshop/summer-school** @AGH?

Thank you for your attention!

acknowledgments: Kamil Kania & the paper co-authors